Rails and Databases

The examples in this book were written using SQLite 3 (version 3.43.2 or thereabouts). If you want to follow along with our code, it's probably simplest if you use SQLite 3 as well. If you decide to use something else, it won't be a major problem. You may have to make minor adjustments to any explicit SQL in our code, but Rails pretty much eliminates database-specific SQL from applications.

If you want to connect to a database other than SQLite 3, Rails also works with MySQL, trilogy, PostgreSQL, Oracle, SQLServer, JDBC My SQL, JDBC Sqlite3, JDBC PostgreSQL, and just plain JDBC. For all but SQLite 3, you'll need to install a database driver—a library that Rails can use to connect to and use with your database engine. This section contains links to instructions on how to get that done.

The database drivers are all written in C and are primarily distributed in source form. If you don't want to bother building a driver from source, take a careful look at the driver's website. Many times, you'll find that the author also distributes binary versions.

If you can't find a binary version, or if you'd rather build from source anyway, you need a development environment on your machine to build the library. For Windows, you need a copy of Visual C++. For Linux, you need `gcc` and friends (but these will likely already be installed).

On OS X, you need to install the developer tools, which come with the operating system but aren't installed by default. You also need to install your database driver into the correct version of Ruby. If you installed your own copy of Ruby, bypassing the built-in one, it's important to have this version of Ruby first in your path when building and installing the database driver. You can use the `which ruby` command to make sure you're *not* running Ruby from `/usr/bin`.